

LLM Inference & Serving

LLM-Agents

Presented by: **Aleksandr Algazinov**



HSE University



Using an LLM effectively is not enough.

The person who uses it effectively loses to the one who uses it effectively **and efficiently**. Today is about the second word — what a call actually costs, and where that cost can be cut.

Motivation

Three problems you will hit, and cannot yet fix.

1 • YOUR LATENCY IS HIGH

The agent works, but every task takes a minute. How do you make the system run faster?

2 • YOU SPENT MOST OF YOUR TOKENS ON THE PROMPT

The bill is dominated by the system prompt, not by the answers. How do you optimise that?

3 • YOU DOWNLOADED A MODEL FROM HUGGING FACE

The weights are on disk. How do you actually serve it — and to more than one user?

One block of today answers each. So does last week's **why cache K and V?**

The vocabulary

TTFT

Time to first token. "Did it hear me?"

TPOT

Time per output token.
Reading speed.

LATENCY

One request, end to end $\approx$
 $\text{TTFT} + \text{output tokens} \times \text{TPOT}$.

THROUGHPUT

Tokens per second across *all* requests. What the operator pays for.

RPS

Requests completed per second. What a load balancer counts — long and short requests are equal here, not in tokens/s.

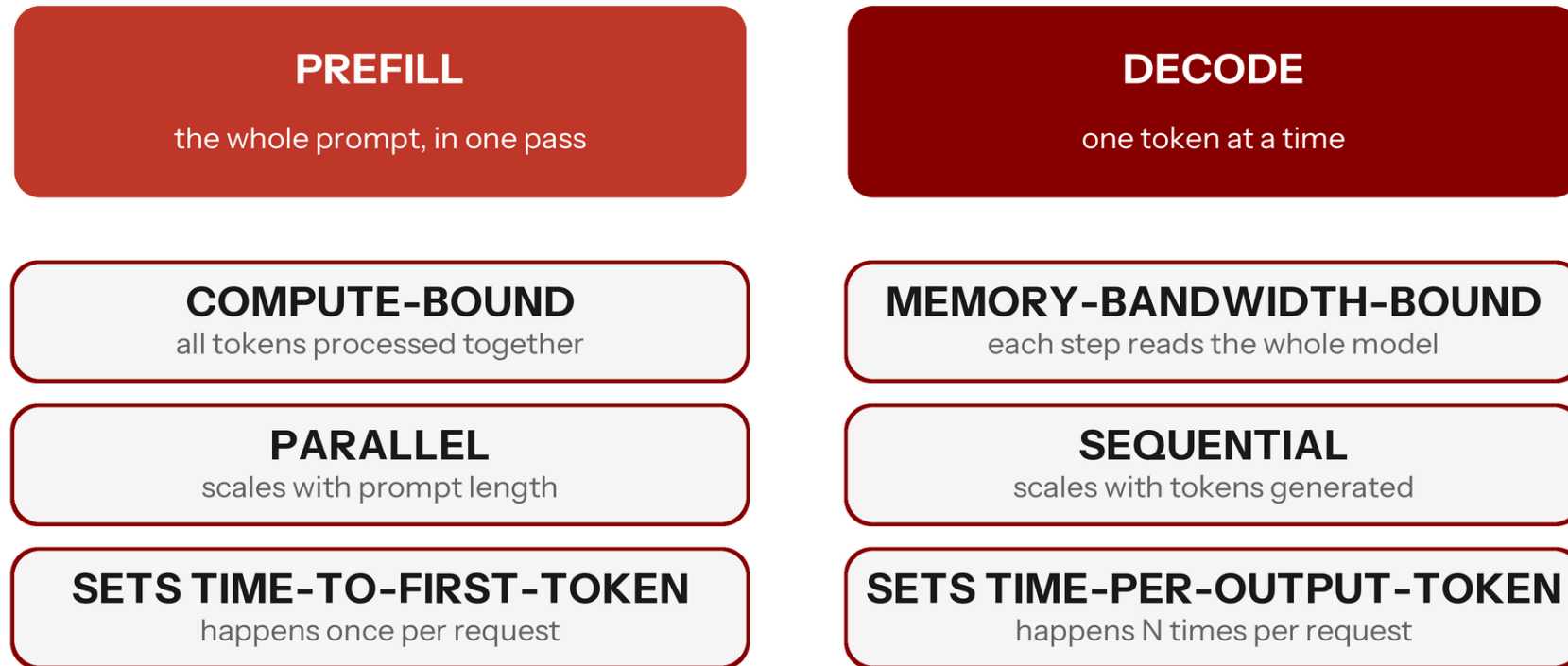
The tension: batching raises throughput and usually **raises latency** per request. Almost every decision today is a point on that trade.

Prefill and Decode

One name, two different machines

The frame everything else uses

A request is not one uniform workload. It is two, with opposite bottlenecks.



batching helps DECODE because the weights are read once per step

two different machines, wearing one name

Why the distinction pays

PREFILL IS COMPUTE-BOUND

All prompt tokens are processed together, so the GPU is doing dense arithmetic. Longer prompt, more compute, but it parallelises.

DECODE IS MEMORY-BOUND

Each step produces *one* token but must read the whole model's weights. The arithmetic is trivial; the memory traffic is not.

This one fact explains batching, the KV cache's cost, and why speculative decoding can work at all.

The KV Cache

The answer to last week's question

Every token gets its own K and V

$$K_i = h_i W_K \qquad V_i = h_i W_V$$

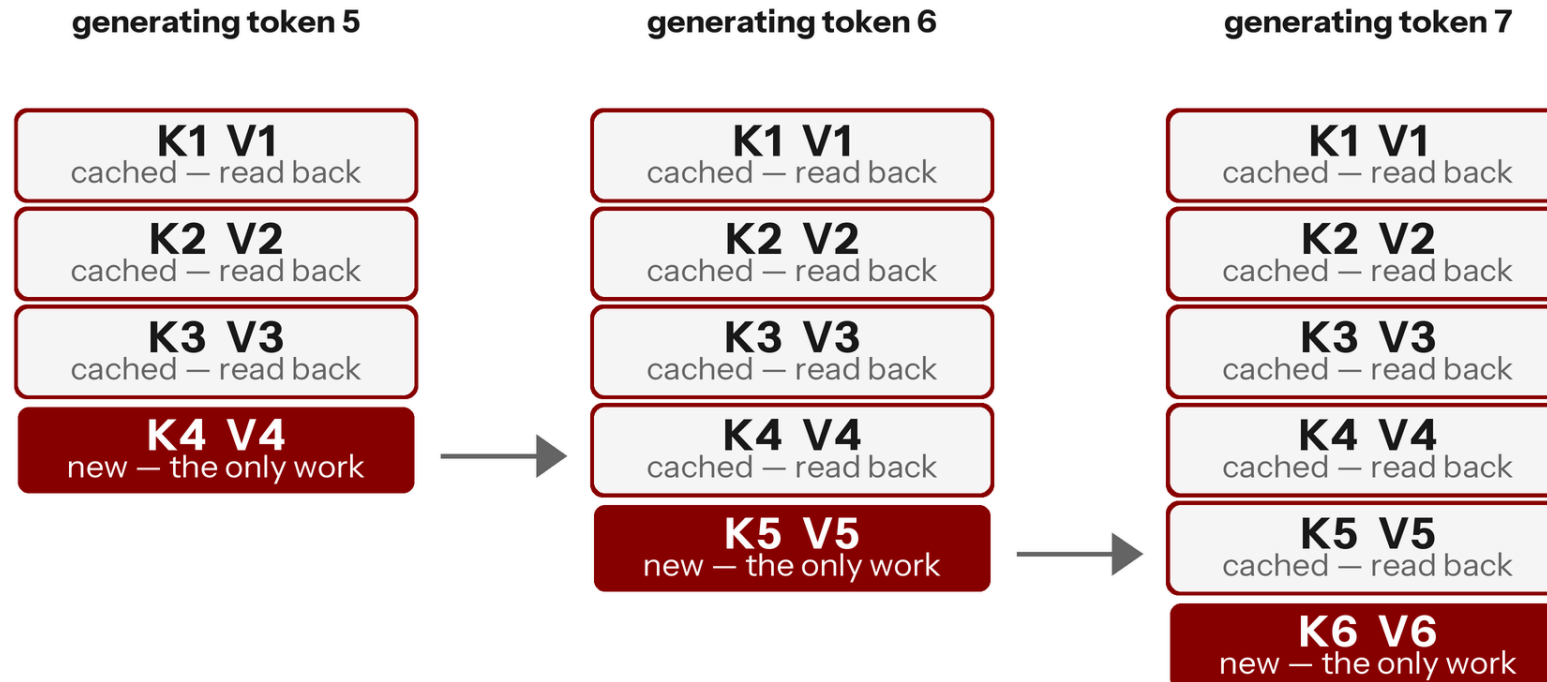
Recap of Lecture 02: one key and one value per token, from that token's hidden state and the frozen weights.

- 1 K_i and V_i depend on token i only** — its own hidden state, and weights that do not change at inference.
- 2 The causal mask** means h_i is computed from tokens 1..i. It is the same number at step i and at step n .
- 3 So K_i , V_i never change once written.** The only new work each step is the newest row.

That is the whole reason a cache is possible.

Watch it grow

Three decode steps. Every row but the newest is a read; only the newest is work.



Cache what cannot change

Earlier tokens' keys and values are **fixed** once computed. Recomputing them is pure waste.

generating token 4

WITHOUT A CACHE



every earlier token's K and V computed again

WITH A CACHE



only the new token does work — the rest are cached

THE COST

cache size grows with $\text{sequence length} \times \text{batch} \times \text{layers} \times \text{heads}$

which is why a server runs out of memory before it runs out of compute

What the cache costs

IT GROWS FAST

Size scales with **sequence length × batch size × layers × heads**. Nothing about it is small.

AND IT IS WHY

- Long contexts cost more than the tokens suggest
- A server hits a memory limit before a compute limit
- How many users you can serve is a memory question

Prefix reuse

If two requests begin with the **same tokens**, that part of the cache can be computed once and shared.

WHERE IT SHOWS UP

A long system prompt, a few-shot preamble, or a document that every question is asked about.

SCOPE NOTE

Here it is the **mechanism**. Designing prompts to hit it — and what that saves — is Lecture 13.

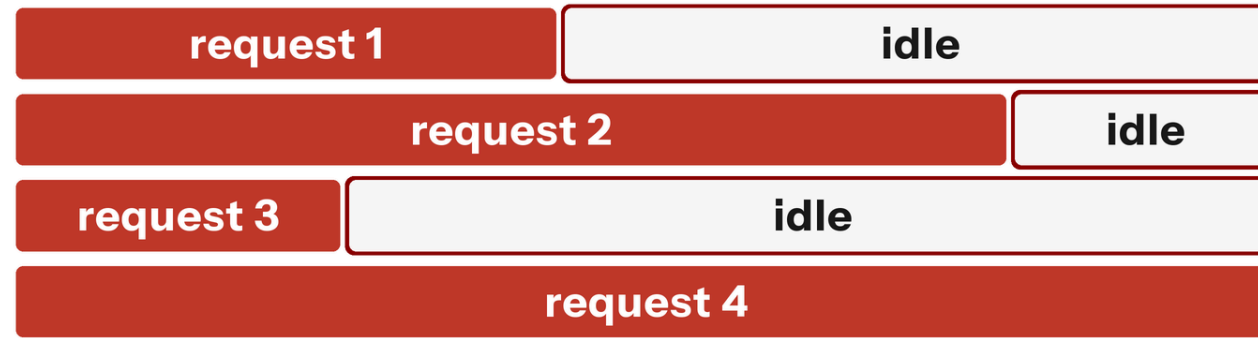
Batching

Where the wasted slots go

Static and continuous

The question is when a finished request gives its slot back.

STATIC BATCHING



the slot is held
until the slowest
request finishes

CONTINUOUS BATCHING



a finished request
frees its slot
immediately

DISCUSSION

You run 32 requests as one batch instead of one at a time. Does each request get roughly 32× slower?

If not — what is being shared between them? And what eventually stops you adding a 33rd?

Decode is waiting on memory, not on maths

WHAT A DECODE STEP ACTUALLY DOES

Read **every weight** in the model out of memory, then do a handful of arithmetic per sequence. Reading 70 GB takes the same time whether you are serving one sequence or fifty.

SO THE GPU IS IDLE

Not idle waiting for work — idle waiting for **memory**. The arithmetic units have nothing to do while the weights arrive. Batching fills that gap with sequences that were going to wait anyway.

WHICH IS WHY THE CEILING IS MEMORY TOO

The 33rd request is refused not because the GPU ran out of compute, but because there is nowhere left to put its KV cache.

Why batching pays at all

THE KEY FACT

In decode, the model's weights are read from memory **once per step** — no matter how many sequences are in the batch. Extra sequences ride along nearly free.

SO THROUGHPUT RISES

Until you run out of memory for the KV cache — which is the real ceiling.

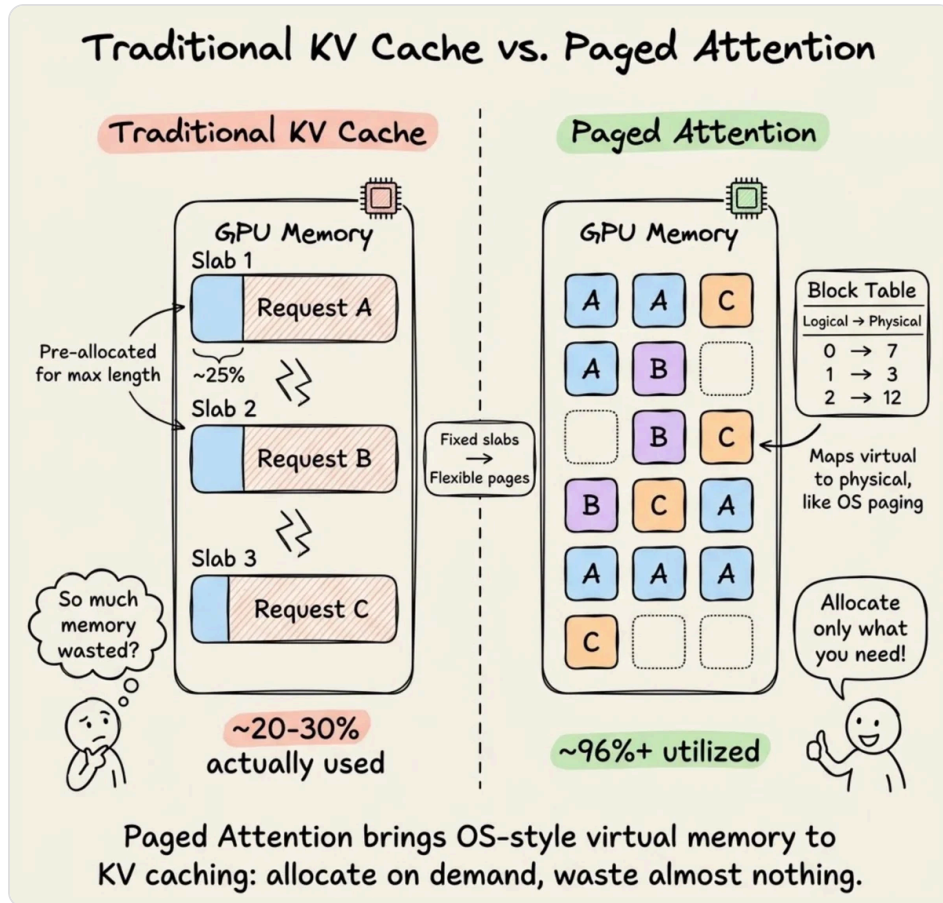
AND LATENCY RISES TOO

Your request now shares a step with others.
Throughput and latency are not the same goal.

PagedAttention

Virtual memory, applied to the cache

Fragmentation, and the fix



SLABS

One contiguous block per sequence, sized for the **maximum** length. Most of it is never used.

PAGES

Fixed-size blocks handed out on demand. A per-sequence **block table** maps logical to physical — exactly OS paging.

THE EFFECT

From roughly a quarter of cache memory holding real tokens to almost all of it.

What that buys

HIGHER OCCUPANCY

Memory is handed out in small blocks as they are needed, so far more sequences fit in the same GPU.

SHARING FOR FREE

Two sequences with the same prefix can point at the *same* physical blocks — prefix reuse, implemented.

This is the core idea in **vLLM**, and the reason it became the default open serving stack.

From Hugging Face to an endpoint

- 1 **Weights** are a Hugging Face repo id. The server downloads them on first start.
- 2 **Start the server.** It exposes an OpenAI-compatible HTTP API.
- 3 **Point any client at it** by changing `base_url`. Your application code does not change.

```
vllm serve Qwen/Qwen2.5-7B-Instruct # vLLM 0.29 – illustrative, not run in class
```

```
client = OpenAI(base_url="http://localhost:8000/v1", api_key="none")  
client.chat.completions.create(model="Qwen/Qwen2.5-7B-Instruct", messages=[...])
```

SGLang, same three steps: `python -m sglang.launch_server --model-path <repo-id>` (SGLang 0.5), port 30000.

Constrained Decoding

Making invalid output impossible

The mechanism

At each step the model produces scores over the whole vocabulary. A constraint **masks the tokens that cannot legally come next** — before sampling.

SO

- Invalid output is **impossible**, not merely unlikely
- The model cannot "forget" the format under pressure
- No parsing-and-retrying after the fact

HOW IT IS EXPRESSED

- A grammar, or a JSON Schema
- Compiled into a state machine over allowed tokens
- Applied as a mask on every step

What it costs

OVERHEAD

Building and applying the mask is work on every token. Usually small, never zero.

CORNERS

A badly specified grammar can force the model into a path it cannot complete sensibly. The output is valid and useless.

WHERE THIS GOES

Lecture 05 uses it to get typed output from tools. Lecture 10 uses a schema to steer *reasoning*. Both rest on what you just saw.

Why long contexts get expensive twice

AT PREFILL

Every token attends to every other, so the work of processing a prompt grows **faster than linearly** with its length.

AT DECODE

The KV cache for that prompt is held in memory for the *whole* generation, taking space another request could have used.

THE PRACTICAL CONSEQUENCE

"Just paste the whole document in" is a decision with a price on both axes. Lecture 07 is about the alternatives.

What you actually control through an API

Most of this lecture describes a server you will not operate. Here is the surface you do touch.

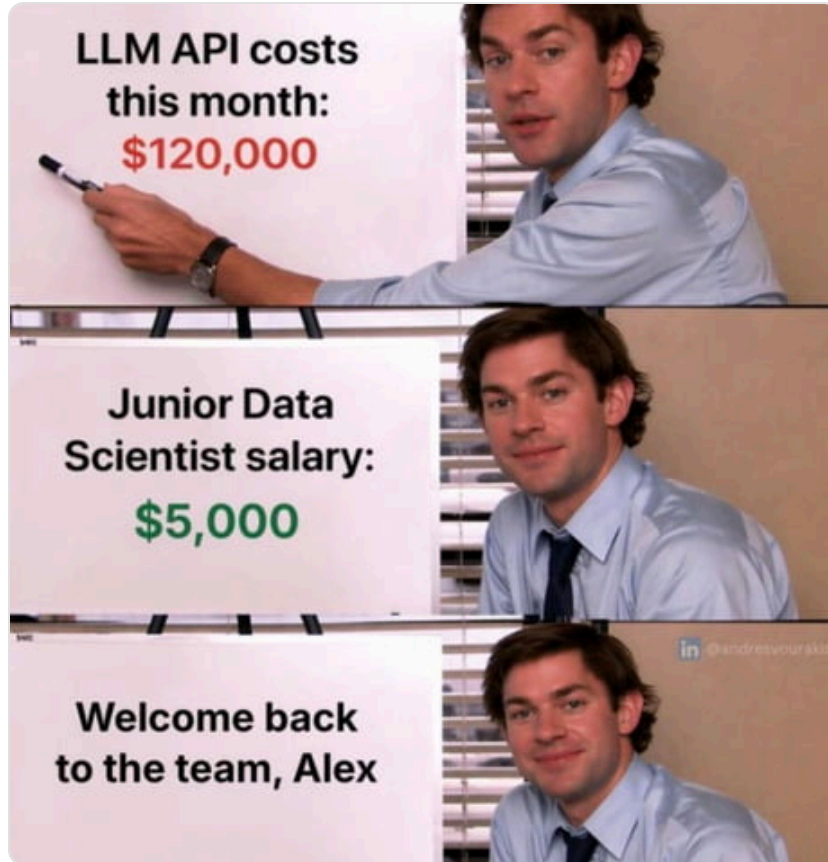
YOU CONTROL

- Prompt length — the biggest lever you have
- A stable prefix, so caching can apply
- `max_tokens`, and stop sequences
- Streaming, for perceived latency

YOU DO NOT CONTROL

- Batch composition — you share with strangers
- Which GPU, or how loaded it is
- Whether your cache entry survived

Hosted API or your own server?



HOSTED API

No infrastructure, no capacity planning. Frontier models you cannot self-host. Per-token pricing that scales with success.

SELF-HOSTED

Data never leaves your infrastructure. Fixed cost once volume is high enough. Any model, any fine-tune, any version.

BE HONEST ABOUT THE CROSSOVER

Self-hosting trades a per-token bill for an **engineering team**. For this course, use the API.

Speculative Decoding

Recognise it, do not master it

DISCUSSION

A small, cheap model drafts the next few tokens. The big model checks them and you keep the ones it agrees with.

It is measurably faster. So what are you giving up in output quality to get that speed?

Nothing — and that is the interesting part

- 1** The draft model proposes k tokens. It is only ever a **guess about what the big model would say** — it never gets to decide anything.
- 2** The big model scores all k positions in **one forward pass**, producing the same distribution it would have produced token by token.
- 3** Each draft token is accepted only if it matches that distribution; the first rejection is resampled to correct for the bias. The result is **provably the large model's own distribution**.

You are not trading quality for latency. You are trading *extra compute* for latency — which decode had spare anyway.

Speculative decoding

THE IDEA

A small draft model proposes several tokens. The large model verifies them **in one pass** and accepts the longest correct prefix.

THE GUARANTEE

Output is distributed identically to the large model alone. This is a latency optimisation, not a quality trade.

It pays when the draft is usually right — and decode being memory-bound is exactly why verifying many tokens at once is nearly free.

Quantization

Fewer bytes per weight

What a number costs

Format	Bits	Sign / exponent / mantissa	70B model, weights only
FP32	32	1 / 8 / 23	280 GB
FP16	16	1 / 5 / 10	140 GB
BF16	16	1 / 8 / 7 — FP32's range, less precision	140 GB
FP8	8	E4M3 or E5M2	70 GB
INT8	8	integer + a scale per group	70 GB
INT4	4	integer + a scale per group	35 GB

WHAT GOES FIRST

Range first (FP16 overflows where BF16 does not — why training uses BF16), then **precision**. The last column is arithmetic, parameters $\times$ bits $\div$ 8, not a benchmark.

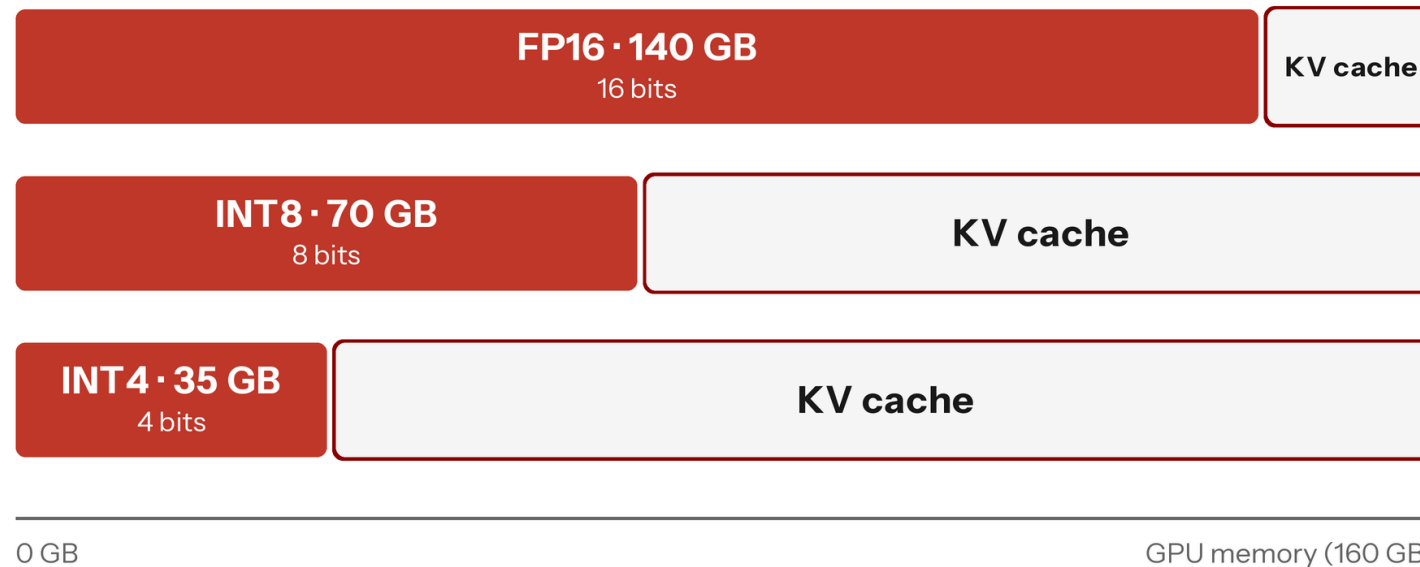
What quantization buys

Decode reads every weight every step: bytes per weight is the lever, for speed **and** for how many fit.

A 70B-PARAMETER MODEL ON THE SAME GPU

weights read from memory on every decode step

left for the KV cache



same model, same GPU: every halving of the weights is time saved per step and sequences gained per batch

Two kinds, one price

WEIGHT-ONLY

Weights stored small, arithmetic still in 16-bit.
The common case for LLM serving: GPTQ, AWQ, GGUF are all here.

WEIGHT-AND-ACTIVATION

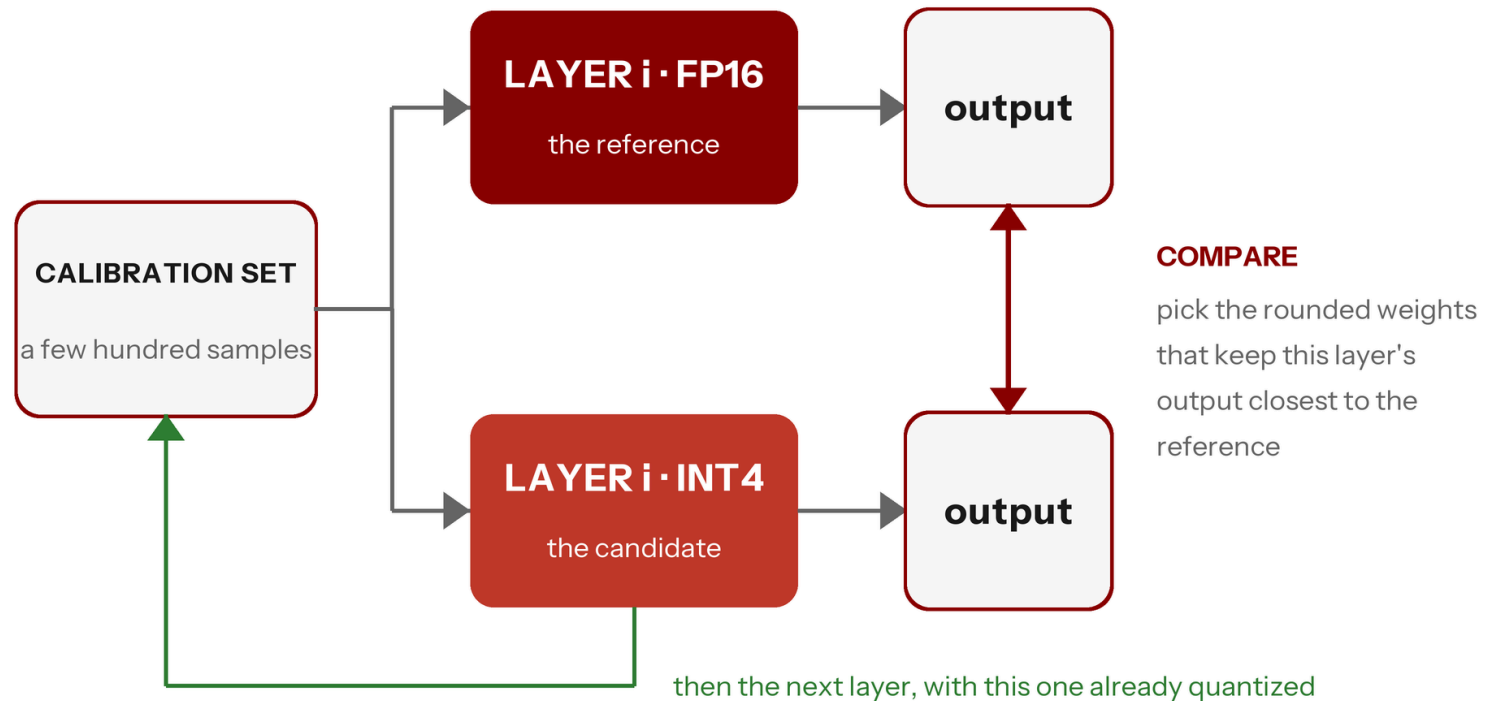
The arithmetic itself runs in INT8 or FP8. Faster still, but it needs hardware that supports it.

THE PRICE

Some quality — measured as a **perplexity gap** or a **task-eval gap** against the full-precision model.
Usually small at 8 bits, noticeable at 4. Never assume zero: measure on your task.

How it is done, roughly

No gradients, no labels. One layer at a time: keep its **output** closest to full precision.



no training, no labels — GPTQ and AWQ differ in which weights are allowed to keep precision

GGUF, GPTQ, AWQ

GGUF

llama.cpp's file format. CPU, Apple silicon, mixed hardware. What Ollama runs.

GPTQ

Post-training, one layer at a time, using **second-order** error information to pick the rounding.

AWQ

Post-training; the few weight channels with the largest **activations** are scaled up before rounding, so they keep precision.

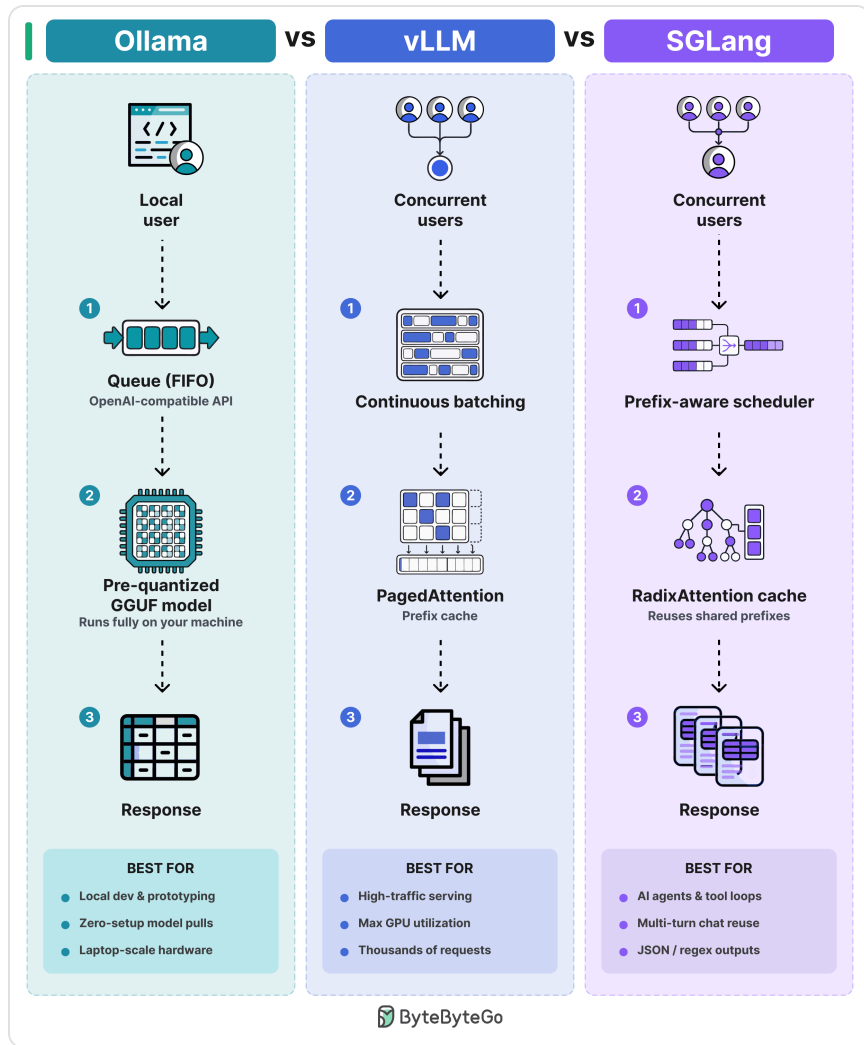
THE PRACTITIONER'S SUMMARY

GGUF for a laptop; a GPTQ or AWQ checkpoint for GPU serving in vLLM or SGLang. Lower precision buys memory and speed and costs some quality — which format you meet depends on the runtime you picked.

Serving Stacks

vLLM and SGLang

Two servers, one job



OLLAMA

One local user, a FIFO queue, a pre-quantized GGUF model. Prototyping, not serving.

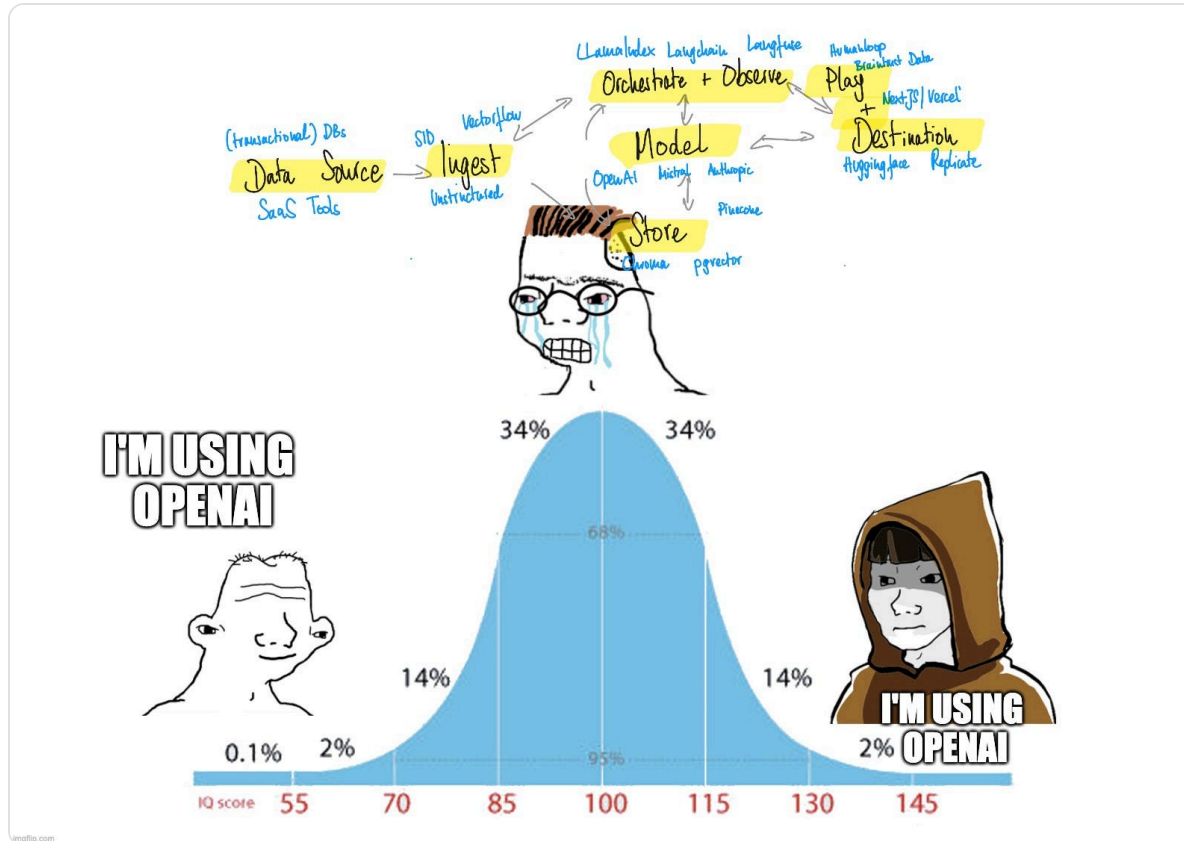
VLLM

Where **PagedAttention** came from. Continuous batching, the widest model and hardware coverage, the default everyone benchmarks against.

SGLANG

RadixAttention: the KV cache in a radix tree keyed by prefix, so any request sharing a prefix reuses it automatically. Both serve the same checkpoints behind an OpenAI-compatible endpoint; SGLang's structured side is Lecture 09.

How to pick



LEAN VLLM WHEN

The model or the hardware is unusual, or you want the largest set of integrations and quantized checkpoints.

LEAN SGLANG WHEN

Prefix sharing dominates the traffic: multi-turn chat, agents re-sending history every step, batch evaluation.

THE HONEST ANSWER

That is a prior, not a decision. Both take the same model with the same three-step launch — a **one-hour benchmark on your own traffic** decides it.

DISCUSSION

Your pipeline runs five times slower than you expected. It makes about ten LLM calls per task.

What do you try first? And what do most people try first?

Cheap levers first

- 1 Measure.** TTFT or TPOT? Which of the ten calls dominates? Ten calls at 0.5 s and one at 20 s are different problems.
- 2 Shrink the prompt** — the system prompt first — and keep the prefix stable so the cache hits.
- 3 Cap `max_tokens`** and stream, so the first token arrives early (Lecture 13 has the rest).
- 4 Parallelise** the calls that do not depend on each other.
- 5 Smaller model** for the easy steps. Routing by difficulty is Lecture 08.
- 6 Change provider or server.** A different API, or a self-hosted stack.
- 7 If you self-host:** quantize, then try speculative decoding.

The order matters. Most teams start at six and skip one.

Summary

COVERED

- TTFT, TPOT, throughput, and the trade
- Prefill compute-bound, decode memory-bound
- The KV cache, its cost, prefix reuse
- Continuous batching; PagedAttention
- Constrained decoding as masking
- Quantization: formats, payoff, calibration
- vLLM vs SGLang; levers for a slow agent

KEEP THESE

- Decode is memory-bound — that explains most of it
- Memory, not compute, is the serving ceiling
- Throughput and latency are different goals
- A constraint at decode beats a parser afterwards
- Fewer bytes per weight: faster decode, more room
- Measure and shrink the prompt before switching servers

References

- Kwon et al. 2023 · **Efficient Memory Management for LLM Serving with PagedAttention** · arXiv 2309.06180
- Yu et al. 2022 · **Orca: A Distributed Serving System for Transformer-Based Generative Models** · OSDI '22
- Leviathan et al. 2023 · **Fast Inference from Transformers via Speculative Decoding** · arXiv 2211.17192
- Willard & Louf 2023 · **Efficient Guided Generation for Large Language Models** · arXiv 2307.09702
- Frantar et al. 2022 · **GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers** · arXiv 2210.17323
- Lin et al. 2023 · **AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration** · arXiv 2306.00978
- Micikevicius et al. 2022 · **FP8 Formats for Deep Learning** · arXiv 2209.05433
- Zheng et al. 2023 · **SGLang: Efficient Execution of Structured Language Model Programs** · arXiv 2312.07104
- Vaswani et al. 2017 · **Attention Is All You Need** · arXiv 1706.03762 — K/V recap

What's Next • Lecture 04: Introduction to Agents

WHAT'S COVERED

- What an agent is — including the non-LLM kind
- Anatomy: model, tools, memory, loop, termination
- ReAct and Plan-and-Execute
- When *not* to use an agent

HOW TODAY CONNECTS

- You know what a call costs, and why
- Constrained decoding returns in Lecture 05
- Prefix reuse returns as economics in Lecture 13

Thank You

Questions & Discussion

Lecture 03 · LLM Agent Development